

项目总结报告

日期：2026.07.30

组号	07	项目名称	GitHub 仓库问答与 Issue 分析系统
编程语言	Python, TypeScript / JavaScript, Java	开发平台和框架	FastAPI, Spring Boot, React (Vite / Tailwind CSS), PostgreSQL (pgvector), Docker Compose
项目工作小结			

1. 项目实现了哪些功能？简要一一列出。

GitHub 身份认证与权限管理：支持基于 GitHub OAuth 的安全登录与用户授权管理。

多仓库与 Issue 管理：支持检索和展示用户 GitHub 仓库列表、查看 Issue 列表及详情。

自动化知识库构建：基于 CodeWiki 自动提取代码的 AST（抽象语法树）结构，生成代码图谱并持久化至 PostgreSQL。

智能问答与 GraphRAG 检索：结合图检索增强生成（GraphRAG）技术，实现基于全局代码上下文的高准确度 Q&A 问答。

智能 Issue 分析与仓库辅助操作：结合大模型能力自动分析 Issue 意图，并辅助完成代码定位与自动化修改建议。

2. 项目实现了哪些非功能？简要一一列出。

高可用容器化部署：全套知识引擎和服务通过 Docker Compose 进行极简部署，实现开箱即用。

高效向量与图索引存储：采用 pgvector 插件优化代码向量检索与图节点响应速度。

系统降级与容错保障：前端与基础 API 具备降级机制，当 Docker 引擎或 CodeWiki 暂未启动时，登录、仓库/Issue 浏览等核心基础功能仍可稳定可用。

微服务架构协同：前端 React 界面与底层 Python 服务、Spring Boot 业务中枢解耦，保障了系统的水平扩展能力。

3. 项目有哪些特点和亮点？例如业务特色、架构风格、设计模式、模型、算法、项目协同等

业务特色：突破了传统单文件 RAG 的局限，AST code graph + GraphRAG，用结构化代码图支撑仓库问答，提升检索与代码上下文相关性，大幅提升了大模型对复杂大型项目全局代码逻辑的理解能力。

架构风格：采用前后端分离的云原生微服务架构，后端包含 FastAPI 算法服务、Spring Boot 业务中枢以及基于 PostgreSQL (pgvector) 的图/向量双引擎存储。

设计模式：大量使用适配器模式（封装 GitHub API）、工厂模式与策略模式（处理不同的 RAG 检索）。

可降级设计：知识引擎、LLM 非单点；无模型/引擎异常时仍可完成基础协作能力。

项目协同：采用 Git Feature Branch（特性分支迭代）开发模式。各服务组件模块化拆分，利用 Docker 和 uv 统一跨平台的依赖和虚拟环境管理。

4. 项目的质量控制如何？简要一一列出。例如软件评审、单元测试、API 测试、系统的功能测试、性能测试、兼容性测试、界面测试等等

软件评审：严格实行 Code Review（代码审查）与 Pull Request (PR) 准入机制（如从 feature/cloud 向主分支合并必须经过全量测试验证与团队评审）。

API 测试：利用 FastAPI 内置的 OpenAPI/Swagger 接口文档及 Postman 进行全面的 RESTful 接口连通性与健康度校验（/api/health）。

系统功能测试：针对 OAuth 授权、仓库拉取、知识库构建、GraphRAG 问答链条进行了全链路整合测试。

兼容性与环境测试：对 Windows (PowerShell/WSL2) 与 Linux 环境下的容器化部署进行自动化脚本与交互兼容性测试。

界面测试：在多种屏幕分辨率下对 React 组件与 UI 交互流进行适配和响应式测试。

5.大模型工具辅助完成了哪些方面的软件开发实践活动？

代码重构与快速原型搭建：辅助生成 React 前端页面交互逻辑、Tailwind CSS 布局及 FastAPI 路由基底。

复杂算法与脚本编写：辅助编写 CodeWiki 中 AST 节点解析逻辑以及 Docker Compose 脚本和依赖部署流程。

报错诊断与环境排查：在 git 分支冲突合并、PowerShell 语法错误、Docker 引擎管道连接失败及 npm/uv 包管理报错时提供快速排查方案。

技术文档撰写：辅助生成项目 README 部署指南、API 规范以及架构设计说明文档。

6. 大作业里，哪些代码开发、任务环节无法由大模型独立实现，需要人机协作或纯人工完成？

系统多服务交互与架构设计（纯人工）：微服务边界划定、多分支 Git 协作策略、PostgreSQL 与引擎间的业务流规划。

复杂 Merge 冲突与环境配置（人机协作）：Git 合并冲突时的代码保留抉择、本地 Docker 与 Windows 管道通信的底层环境调优。

GraphRAG 提示词工程与领域效果调优（人机协作）：针对代码分析场景调优 Prompts，消除大模型幻觉，使其精准输出结构化 AST 节点。

全链路业务逻辑端到端联调（纯人工）：跨前端、Spring Boot 中转层与 Python 算法层的整体联调与边界异常兜底处理。

项目组成员以及大模型工具对项目的贡献度（%）							
	姓名	需求	设计	编码	测试	项目管理*	小计
	蔡远哲	20	20	25	20	20	21%
	代品	20	25	25	25	20	23%
	李明睿	20	20	15	15	20	18%
	林圣	20	20	20	20	20	20%
	蒋一哲	20	15	15	20	20	18%
注： 同学们应理解与评审大模型生成的内容，并负全部责任；项目管理包括项目计划、报告、沟通与协调等。							
经验、教训和建议							
大模型辅助开发经验：在定义明确、边界清晰的模块（如 API 接口声明、前端组件生成、正则表达式撰写）中，AI 能够提升 300% 以上的效率。提问时提供精准的错误日志（如 <code>git error</code> 或 <code>npm ENOENT</code>）能让 AI 秒级给出解决方案。 教训与踩坑： 盲目依赖配置导致路径偏差：AI 生成的说明文档可能假设了单包目录结构，但实际项目中前后端独立分割，导致直接运行脚本时找不到 <code>package.json</code> 或 <code>pyproject.toml</code>。需时刻保持对项目实际物理目录结构的清晰认知。 环境依赖敏感：如 Docker 引擎未启动或 Git 分支未完全同步时，AI 可能会生成多余的修补代码，本质问题其实只是本地服务未正常拉起。 给后续开发者的建议：在多分支协同开发时，应建立规范的 PR 准入与自动化 CI/CD 流程；在接入大模型生成代码前，先由人工定好数据结构和架构图，再将拆解后的具体子任务交给 AI 实现，这样能最大化人机协作效能。							

项目组各成员签字：